# Class Definition

class ‹*scope*::› *classname* ‹ < *superexpr* ›
  *body*
end

class << *obj*
  *body*
end

A Ruby class definition creates or extends an object of class Class by executing the code in body. In the first form, a named class is created or extended. The resulting Class object is assigned to a constant named classname (keep reading for scoping rules). This name should start with an uppercase letter. In the second form, an anonymous (singleton) class is associated with the specific object.

If present, superexpr should be an expression that evaluates to a Class object that will be the superclass of the class being defined. If omitted, the superclass defaults to class Object.

Within body, most Ruby expressions are executed as the definition is read. However:

- Method definitions will register the methods in a table in the class object.
- Nested class and module definitions will be stored in constants within the class, not as global constants. These nested classes and modules can be accessed from outside the defining class using :: to qualify their names.

```
module NameSpace
  class Example
    CONST = 123
  end
end

obj = NameSpace::Example.new

a = NameSpace::Example::CONST
```

- Calling the include method with modules as arguments will add those modules as anonymous superclasses of the class being defined.

The classname in a class definition may be prefixed by the names of existing classes or modules using the scope operator (::). This syntax inserts the new definition into the namespace of the prefixing module(s) and/or class(es) but does not interpret the definition in the scope of these outer classes. A classname with a leading scope operator places that class or module in the top-level scope.

In the following example, class C is inserted into module A's namespace but is not interpreted in the context of A. As a result, the reference to CONST resolves to the top-level constant of that name, not A's version. We also have to fully qualify the singleton method name, because C on its own is not a known constant in the context of A::C.

```
CONST = "outer"

module A
  CONST = "inner"   # This is A::CONST
end
```